

UT3 – Programación de comunicaciones en red – EJERCICIOS 7 – 8 – 9 – 10

Programación de comunicaciones en red

EJERCICIOS 7 – 8 – 9 – 10



EJERCICIO 7: Juego “Ahorcado”

Objetivo General

Crear una aplicación de red en Java que implemente el clásico juego del **Ahorcado**. El **servidor** mantiene la lógica de la palabra secreta y la gestión de intentos, mientras que el **cliente** se encarga de interactuar con el usuario.

Puntos clave

1. Lógica del juego:

- El **servidor** genera o almacena una **palabra secreta**. Por ejemplo, puede ser “PROGRAMACION”, “INTERFAZ”, “SOCKET”, etc. Se podría escoger aleatoriamente de una lista de palabras.
- Se establecen un número de **intentos máximos** (por ejemplo, 6). Si el cliente agota los intentos sin adivinar la palabra, pierde.
- Tras cada **letra** enviada por el cliente, el servidor responde con el **estado actual** de la palabra (p. ej. “P _ O G _ A M A C I O N”) o con algún indicador de fallos.

2. Servidor:

- De nuevo, el servidor arranca un `ServerSocket` y espera conexiones (`accept()`).
- Al entrar un cliente, se abre un `Socket` y se inician los flujos (`InputStream`, `OutputStream`).
- La **lógica**:
 - Se envía un mensaje inicial con los **intentos permitidos** o la **longitud de la palabra**.
 - Se mantienen variables internas para saber cuántas letras acierta el cliente y cuántos intentos le quedan.
 - Cada vez que el servidor recibe una letra, actualiza el “estado” y responde, por ejemplo, con “¡Acerto!” o “¡Fallaste!” + estado parcial de la palabra.

3. Cliente:

- Se conecta al servidor por `Socket`.
- Lee y muestra la **situación** del juego (los guiones, los aciertos, cuántos intentos quedan).
- Permite al usuario introducir **letras** (una a la vez) y las envía al servidor.
- Cuando recibe la respuesta “¡Acertaste!” o “Se acabaron tus intentos, la palabra era: ...”, finaliza el juego.

4. Pistas Teóricas:

- Uso de `Random` (si se elige la palabra al azar).
- Bucles de lectura de línea (`in.readLine()`) y de escritura (`out.println()`).
- Un posible estado interno: un array de caracteres (`char[] progreso`), donde se reemplazan los guiones _ por la letra adivinada.
- Control de mayúsculas/minúsculas para comparar correctamente.

UT3 – Programación de comunicaciones en red – EJERCICIOS 7 – 8 – 9 – 10

5. Sugerencias didácticas:

- Trabajar con **control de errores** en caso de que el cliente envíe más de una letra.
- Permitir “abandonar” introduciendo un comando especial.
- Posiblemente, un servidor con varios clientes a la vez: cada cliente juega su propia partida (manejo multi-hilo).

Esqueleto de Código

Servidor (ServidorAhorcado.java)

```
public class ServidorAhorcado {  
    public static void main(String[] args) {  
        // 1. Crear un ServerSocket (puerto 5006, por ejemplo).  
        // 2. Bucle infinito que acepte conexiones.  
        // 3. Para cada cliente:  
        //     a) Generar o escoger la palabra secreta.  
        //     b) Enviar el número de intentos máximos.  
        //     c) Leer letra por letra que el cliente envía.  
        //     d) Actualizar el estado (guiones o letras descubiertas).  
        //     e) Responder con "Acierto", "Fallaste", etc., más el estado actual.  
        //     f) Si agota intentos o completa la palabra, acabar la partida o  
        reiniciar.  
        // 4. Cerrar los recursos al terminar la partida.  
    }  
}
```

Cliente (ClienteAhorcado.java)

```
public class ClienteAhorcado {  
    public static void main(String[] args) {  
        // 1. Conectarse al servidor con un Socket (localhost, 5006).  
        // 2. Esperar y mostrar mensajes de bienvenida (intentos disponibles).  
        // 3. En bucle, solicitar al usuario una letra.  
        // 4. Enviar la letra al servidor, leer respuesta:  
        //     - "Acierto", "Fallaste", mostrar guiones  
        //     - "¡Acertaste!" o "Se acabaron los intentos" -> terminar juego  
        // 5. Cerrar recursos.  
    }  
}
```

UT3 – Programación de comunicaciones en red – EJERCICIOS 7 – 8 – 9 – 10

EJERCICIO 8: Chat Multi-salas (“Grupos de Estudio”)

Objetivo General

Ampliar el concepto de chat para soportar **múltiples salas** o canales de conversación. Cada **cliente** elige a qué sala unirse y solo recibe los mensajes de esa sala.

Puntos Clave

1. Servidor:

- Escucha en un puerto con `ServerSocket`.
- Acepta conexiones, crea un **hilo** por cliente (multi-threading).
- Mantiene una **estructura de datos** (por ejemplo, un `Map<String, List<PrintWriter>>`) donde la **clave** es el nombre de la sala (ej. “Matemáticas”, “Historia”, “Inglés”) y el **valor** es la lista de clientes (sus `PrintWriter`) que están en esa sala.
- Cuando un cliente envía un mensaje, el servidor lo reenvía **solo** a los miembros de la misma sala.
- El servidor podría permitir **cambiar** de sala con un comando especial (ej. `/cambiar SalaX`).

2. Cliente:

- Tras conectarse, el servidor le solicita (o el cliente introduce por su cuenta) a qué sala unirse.
- El cliente puede escribir mensajes que se difunden solo en la sala actual.
- Puede haber un **comando especial** para cambiar de sala (por ejemplo `/cambiar Ingles`).
- Mantener un hilo que reciba los mensajes del servidor y un bucle que envíe los del usuario.

3. Pistas Teóricas:

- Sincronizar el acceso a la estructura compartida (el `Map`) si varios hilos añaden o quitan clientes simultáneamente.
- Añadir un método `broadcast(String sala, String mensaje)` que recorra la lista de `PrintWriter` de esa sala y envíe el mensaje.
- Incluir métodos para **unirse** a una sala (añadir el `PrintWriter` a la lista) y **salir** de la sala anterior (removerlo).

4. Sugerencias didácticas:

- Manejar notificaciones: “Un usuario se ha unido a la sala X”, “Un usuario ha salido de la sala X”.
- Dividir la aplicación en más de una clase: `ServidorChatMultiSalas`, `ManejadorCliente`, etc.
- Explorar un interfaz de consola vs. interfaz gráfica (Swing, JavaFX, etc.) para la experiencia de chat.

UT3 – Programación de comunicaciones en red – EJERCICIOS 7 – 8 – 9 – 10

Esqueleto de Código

Servidor (ServidorChatMultiSalas.java)

```
public class ServidorChatMultiSalas {  
    // Ejemplo de estructura: private static Map<String, List<PrintWriter>> salas =  
    ...  
  
    public static void main(String[] args) {  
        // 1. Crear ServerSocket (puerto 5007).  
        // 2. Bucle infinito con accept().  
        // 3. Por cada cliente, crear un hilo que:  
        //     a) Pregunte/reciba la sala elegida.  
        //     b) Registre el PrintWriter en la sala correspondiente.  
        //     c) Lea mensajes y haga broadcast a la sala.  
        //     d) Cambie de sala si el usuario lo solicita.  
        // 4. Manejo de sincronización al modificar "salas".  
    }  
  
    // Método broadcast(sala, mensaje): recorre la lista de la sala y envía mensaje a  
    // cada PrintWriter  
    // Método cambiarSala(...): elimina de la sala anterior y añade a la nueva sala  
}
```

Cliente (ClienteChatMultiSalas.java)

```
public class ClienteChatMultiSalas {  
    public static void main(String[] args) {  
        // 1. Conectarse al servidor (localhost, 5007).  
        // 2. Introducir/indicar la sala a la que unirse.  
        // 3. Hilo que lee mensajes del servidor y los muestra.  
        // 4. Bucle que lee la consola local y envía mensajes.  
        // 5. Soportar comando "/cambiar SalaX" para cambiar de sala.  
    }  
}
```

EJERCICIO 9: Sistema de Turnos en Línea (“Cola Virtual”)

Objetivo General

Simular un **sistema de turnos** (como en un banco o clínica), donde cada **cliente** que se conecta recibe un número de turno y el servidor notifica cuando llega su turno.

Descripción Didáctica y Aspectos Clave

1. Servidor:

- Puede mantener un **contador global** de turnos (p. ej., un `AtomicInteger` o `int` sincronizado).
- Cada cliente que se conecte incrementa el contador y obtiene su **número de turno**.
- Se simula la “atención” de cada turno. Por ejemplo, se puede implementar un **tiempo de espera** para atender cada turno en orden.
- En un escenario más complejo, el servidor podría hacer **broadcast** a todos los clientes cada vez que pasa al siguiente turno, o podría notificar únicamente al cliente cuyo turno está siendo atendido.

2. Cliente:

- Se conecta, recibe un **turno** (por ejemplo, “Tu turno es 5”).
- Luego, el servidor podría mantener la conexión abierta y, cuando llegue el turno 5, enviar “¡Es tu turno!” al cliente.
- El cliente muestra mensajes de espera o confirmación.
- Puede cerrar la conexión cuando termine su turno.

3. Pistas Teóricas:

- Uso de `AtomicInteger` es útil para **incrementar** el contador de turnos sin problemas de concurrencia.
- Controlar las notificaciones:
 - O un **hilo** en el servidor que va “atendiendo” turnos cada X segundos y notifica a los clientes.
 - O el servidor atiende el turno en el orden de llegada, manteniendo la conexión con un único cliente a la vez.

4. Sugerencias didácticas:

- Se puede experimentar con la **creación de varios “puestos”** de atención en paralelo.
- Permitir que el cliente compruebe su turno de vez en cuando en lugar de esperar pasivamente (polling).
- Manejar la desconexión prematura de clientes o reintentos.

UT3 – Programación de comunicaciones en red – EJERCICIOS 7 – 8 – 9 – 10

Esqueleto de Código

Servidor (ServidorColaTurnos.java)

```
public class ServidorColaTurnos {  
    public static void main(String[] args) {  
        // 1. Crear ServerSocket (puerto 5008).  
        // 2. Mantener un contador global de turnos (ej: AtomicInteger turnoActual =  
new AtomicInteger(1)).  
        // 3. En un bucle accept(), cada cliente que llega se le asigna el valor  
actual del contador y se incrementa.  
        // 4. Podría iniciar un hilo que simule la "atención" (Thread.sleep(...)) para  
simular un tiempo).  
        // 5. Cuando corresponde el turno de un cliente, se le notifica "¡Es tu  
turno!".  
        // 6. Cerrar la conexión tras la atención o según la lógica deseada.  
    }  
}
```

Cliente (ClienteColaTurnos.java)

```
public class ClienteColaTurnos {  
    public static void main(String[] args) {  
        // 1. Conectarse al servidor (localhost, 5008).  
        // 2. Leer el turno asignado.  
        // 3. Esperar notificaciones del servidor (p.ej. "Todavía no es tu turno",  
"¡Es tu turno!").  
        // 4. Mostrar al usuario el momento en que le toca, y luego cerrar.  
    }  
}
```

EJERCICIO 10: Juego Colaborativo “Carrera Virtual de Coches”

Objetivo General

Crear un **juego colaborativo** donde varios clientes controlan sus “coches” en una “carrera” virtual. El servidor actualiza las posiciones y notifica a todos del estado general.

Descripción Didáctica y Aspectos Clave

1. Servidor:

- Mantiene un `ServerSocket`.
- Para cada cliente que se conecta, asigna un **ID** y una **posición inicial** (por ejemplo, 0).
- Guarda las posiciones en una estructura, por ejemplo, `Map<Integer, Integer>` (ID del cliente -> posición).
- Recibe “comandos” del cliente (por ejemplo, “MOVE FORWARD”, “TURN LEFT” si hubiera dirección, etc.).
- Actualiza la posición y envía un **broadcast** del estado global (“Jugador 1: X metros, Jugador 2: Y metros...”).
- Se define una **meta** (p. ej. 50 metros). Cuando un jugador la alcanza, el servidor anuncia “¡Jugador X ganó!” y puede resetear o terminar el juego.

2. Cliente:

- Se conecta, obtiene un ID y la meta de la carrera.
- Un hilo se dedica a **recibir** y **mostrar** las actualizaciones del servidor (estado de todos los jugadores).
- El usuario ingresa **comandos** (por ejemplo, “MOVE FORWARD”) para avanzar.
- Cuando recibe “¡Ganaste!” o “¡Jugador X ganó!”, el cliente finaliza o vuelve a menú.

3. Pistas Teóricas:

- Manejo de un **broadcast** a todos los clientes cada vez que cambian las posiciones. Similar a los chats, pero enviando estados de juego.
- Se pueden usar `ConcurrentHashMap` o `Collections.synchronizedMap` para evitar problemas de concurrencia al actualizar posiciones.
- Manejo de final de partida. ¿Se cierra la conexión o se reinicia la carrera?

4. Sugerencias Didácticas:

- Añadir variables de tiempo o “velocidad” para cada coche.
- Controlar el **orden de llegada** y anunciar 1er lugar, 2do lugar, etc.
- Considerar el caso en que un jugador se desconecte en mitad de la carrera.

UT3 – Programación de comunicaciones en red – EJERCICIOS 7 – 8 – 9 – 10

Esqueleto de Código

Servidor (ServidorCarreraVirtual.java)

```
public class ServidorCarreraVirtual {
    // Posibles estructuras: Map<Integer, Integer> posiciones, List<PrintWriter>
    clientes, int META = 50;

    public static void main(String[] args) {
        // 1. Crear ServerSocket (puerto 5009).
        // 2. Bucle aceptando conexiones. Cada cliente recibe un ID (por ej.,
        socket.getPort() o un contador).
        // 3. Hilo para manejar cada cliente:
        //     - Almacena su posición inicial (0) en un Map.
        //     - Lee comandos, p.ej. "MOVE FORWARD" -> actualiza posición +5
        //     - Hace broadcast de las posiciones a todos los clientes
        //     - Si la posición >= META -> anunciar ganador
        // 4. Manejo de desconexiones (remove del Map, etc.).
    }
}
```

Cliente (ClienteCarreraVirtual.java)

```
public class ClienteCarreraVirtual {
    public static void main(String[] args) {
        // 1. Conectarse al servidor (localhost, 5009).
        // 2. Hilo para recibir y mostrar actualizaciones del servidor (estado de
        todos).
        // 3. Bucle para introducir comandos ("MOVE FORWARD", "SALIR", etc.).
        // 4. Manejar el mensaje de victoria o final de carrera.
        // 5. Cerrar recursos.
    }
}
```

Resumen

1. **Ahorcado:** Combina lógica de juego y comunicación simple (un cliente/servidor por partida o multi-hilo).
2. **Chat Multi-salas:** Manejo de **múltiples grupos** con hilos concurrentes y estructuras de datos más elaboradas.
3. **Cola Virtual:** Simulación de **sistema de turnos**, donde se profundiza en la sincronización de un **contador global** y las notificaciones a clientes.
4. **Carrera Virtual:** Ejemplo de **juego colaborativo** con actualizaciones del estado y mecanismos de **difusión** (broadcast).